

MẠNG NƠ-RON

TS. Hoàng Quốc Việt



Nội dung



Giới thiệu



Một số khái niệm



Phát triển mạng nơ-ron

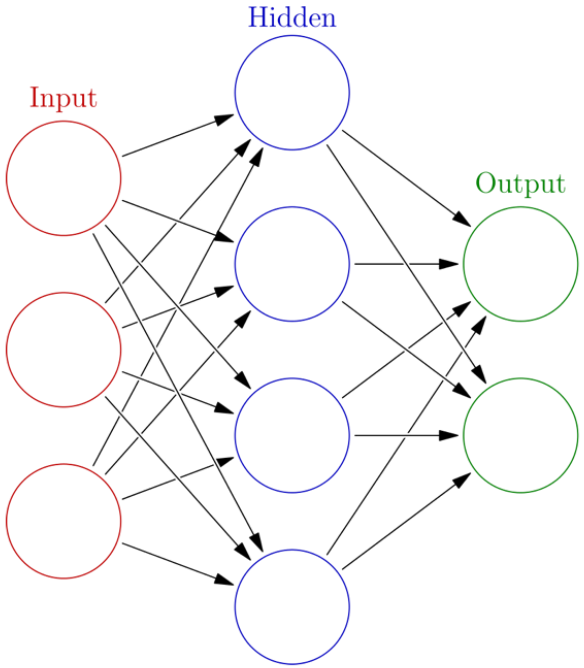


Xây dựng mạng nơ-ron

I. Giới thiệu

- Mạng nơ-ron nhân tạo (Artificial Neural Networks – ANN) là một mô hình tính toán được lấy cảm hứng từ cấu trúc và chức năng của bộ não con người.





I. Giới thiệu

- ANN bao gồm nhiều “nơ-ron nhân tạo” (Artificial Neural) được kết nối với nhau.

- Ứng dụng:
  - Thị giác máy tính.
  - Xử lý ngôn ngữ tự nhiên.
  - Y học và nhiều ứng dụng công nghệ khác.

II. Một số khái niệm

- **Lớp (Layer):**
  - Một mạng nơ-ron bao gồm ba loại lớp (*layer*): Lớp đầu vào (*input layer*), lớp ẩn (*hidden layer*) và lớp đầu ra (*output layer*).
  - Một mạng nơ-ron nhân tạo có thể có nhiều lớp ẩn ở giữa lớp đầu vào và lớp đầu ra.
  - Các lớp ẩn theo thứ tự từ lớp đầu vào đến lớp đầu ra được đánh số thứ thự là *lớp ẩn 1, lớp ẩn 2*…Số lượng lớp trong một mạng nơ-ron nhân tạo được tính bằng số lớp ẩn cộng với 1.

II. Một số khái niệm

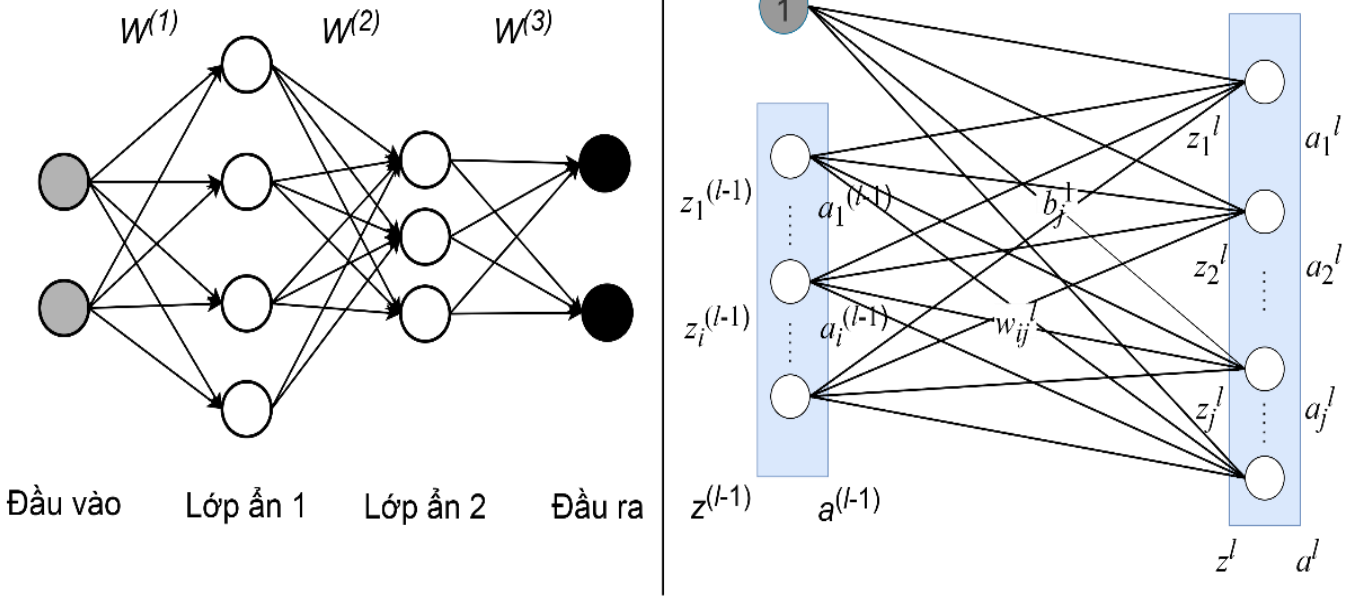
- **Lớp (Layer):**
  - Trong mạng nơ-ron, lớp đầu vào cung cấp dữ liệu ban đầu từ bên ngoài vào mạng mà không có bất kỳ phép tính nào.
  - Các lớp ẩn chịu trách nhiệm thực hiện phép tính trên dữ liệu đầu vào thông qua các nơ-ron ẩn và truyền kết quả đến lớp đầu ra.
  - Lớp đầu ra lấy kết quả từ lớp ẩn cuối cùng làm dữ liệu đầu vào và sử dụng các nơ-ron của nó để tính toán và tạo ra kết quả cuối cùng của mạng.

II. Một số khái niệm

- **Nơ-ron:**
  - Một node hình tròn trong một lớp được gọi là một nơ-ron (hay một đơn vị/một unit).
  - Các nơ-ron của lớp đầu vào, lớp ẩn và lớp đầu ra lần lượt được gọi là *nơ-ron đầu vào, nơ-ron ẩn* và *nơ-ron đầu ra.*
  - Đầu vào của các hidden layer được ký hiệu bởi z, đầu ra của mỗi nơ-ron thường được ký hiệu là a (thể hiện activation function).
  - Đầu ra của nơ-ron thứ *i* trong layer thứ *l* được ký hiệu là $a_i^{(l)}$.

II. Một số khái niệm

- Nơ-ron:

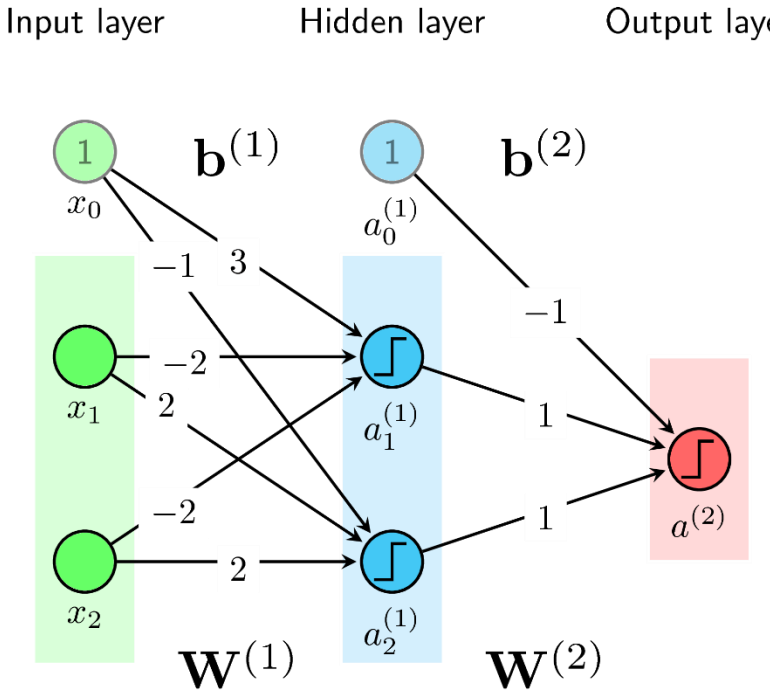


II. Một số khái niệm

- **Trọng số và độ lệch**:
  - Một mạng nơ-ron nhân tạo có *L* lớp sẽ có *L* ma trận trọng số (*Weight*).
  - Các ma trận này được ký hiệu là $\boldsymbol{W}^{(l)}$, *l = 1 , 2 , … , L*. $\boldsymbol{W}^{(l)}$ thể hiện các kết nối từ lớp thứ (*l – 1*) tới lớp thứ *l.*
  - Phần tử $w_{ij}^{(l)}$ thể hiện kết nối từ nơ-ron thứ *i* của lớp thứ (*l − 1*) tới nơ-ron thứ *j* của lớp thứ (*l*) .
  - Các độ lệch (*bias*) của layer thứ (*l*) được ký hiệu là $\boldsymbol{b}^{(l)}$.
  - Tập hợp các weights và biases lần lượt được ký hiệu là ***W*** và ***b***.

II. Một số khái niệm

- **Trọng số và độ lệch:**



$$\mathbf{W}^{(1)} = \begin{bmatrix} -2 & 2 \\ -2 & 2 \end{bmatrix}; \quad \mathbf{b}^{(1)} = \begin{bmatrix} 3 \\ -1 \end{bmatrix}$$

$$\mathbf{W}^{(2)} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}; \quad \mathbf{b}^{(2)} = [-1]$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}; \mathbf{a}^{(1)} = \begin{bmatrix} a_1^{(1)} \\ a_2^{(1)} \end{bmatrix}$$

$$\mathbf{a}^{(1)} = f\left(\mathbf{W}^{(1)T}\mathbf{x} + \mathbf{b}^{(1)}\right)$$

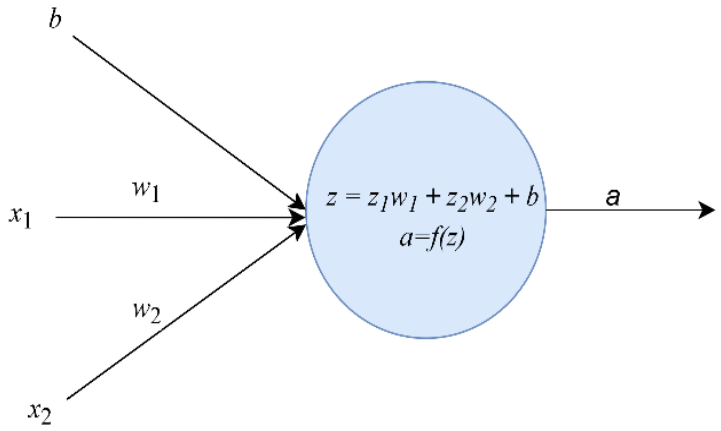
$$a^{(2)} = f\left(\mathbf{W}^{(2)T}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}\right)$$

$$f(.) = \text{sgn}(.) \text{ (element-wise)}$$

II. Một số khái niệm

- **Hàm kích hoạt**:
  - Mỗi output của một nơ-ron (trừ các nơ-ron đầu vào) được tính theo công thức sau: $a_i^{(l)} = f\left(w_i^{(l)\mathrm{T}}\, a^{(l-1)} + b_i^{(l)}\right)$

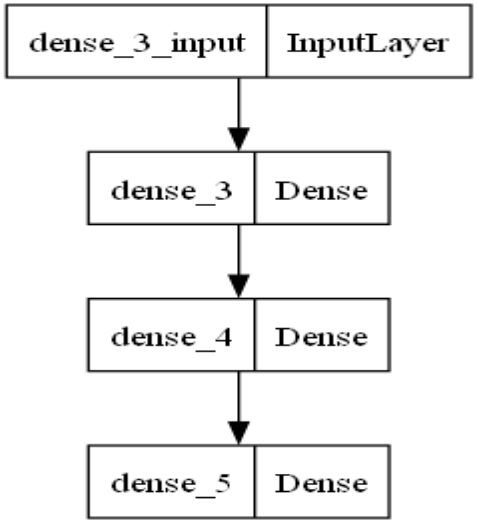
- Các hàm tính toán phổ biến: Sigmoised, tanh, ReLU



II. Một số khái niệm

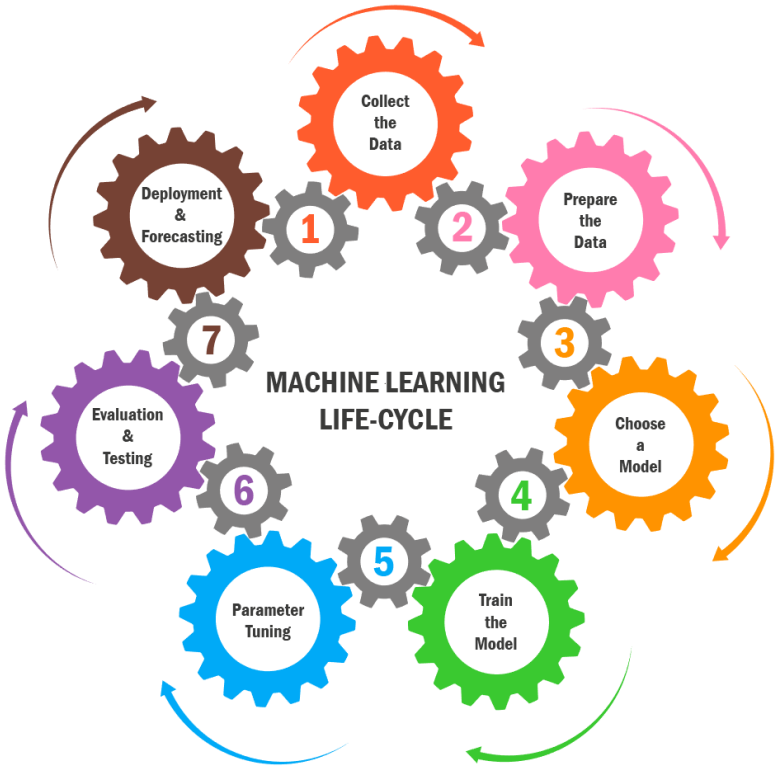
- **Ví dụ:**

```
# Định nghĩa mô hình tuần tự Keras (Cách 1)
model = keras.Sequential(name='Sequential')
# Add 2 lớp ẩn
model.add(layers.Dense(64, activation='relu', input_shape=(784,)))
model.add(layers.Dense(32, activation='relu'))
# Lớp đầu ra là lớp cuối cùng của
model.add(layers.Dense(10, activation='softmax'))
```



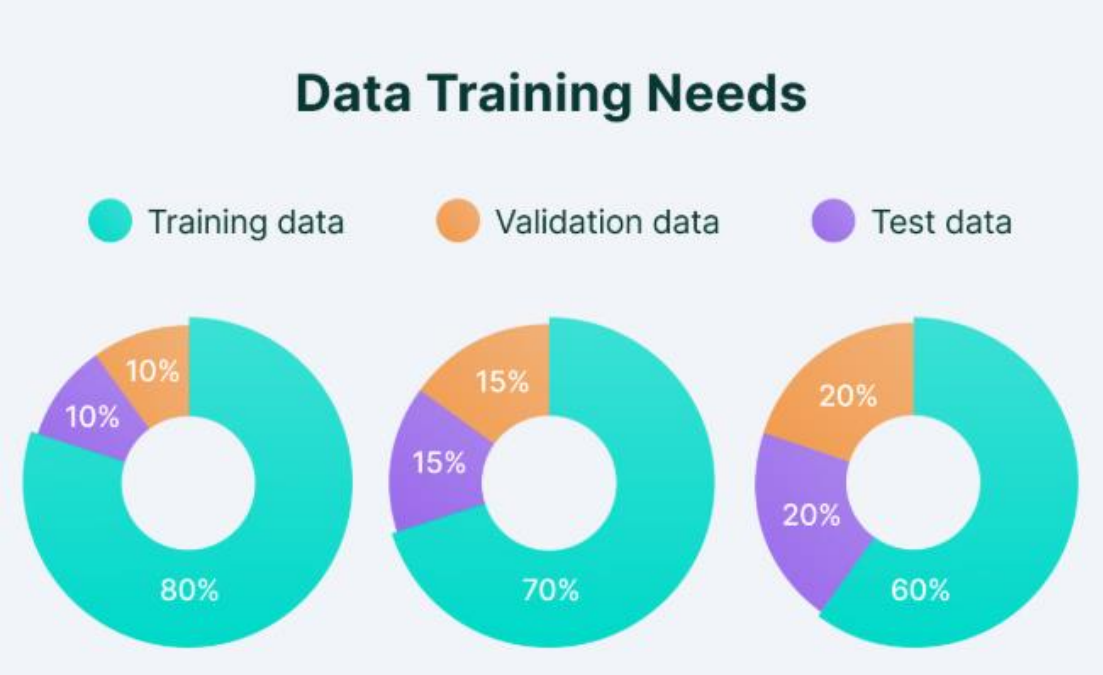
III. Phát triển mạng nơ-ron

- Bước 1: Chuẩn bị dữ liệu.
- Bước 2: Xây dựng mạng nơ-ron.
- Bước 3: Lựa chọn hàm mất mát và tối ưu hoá.
- Bước 4: Huấn luyện mô hình.
- Bước 5: Đánh giá và triển khai mô hình.



III. Phát triển mạng nơ-ron

- Bước 1: Chuẩn bị dữ liệu:
  - Bộ dữ liệu dùng huấn luyện mạng nơ-ron được chia thành ba loại khác nhau: Dữ liệu huấn luyện (*training data*), dữ liệu xác thực (*validation data*) và dữ liệu kiểm tra (*testing data*).



III. Phát triển mạng nơ-ron

- Bước 1: Chuẩn bị dữ liệu:

```
# BƯỚC 1: ĐỌC DỮ LIỆU TỪ FILE diabetes.csv VÀ CHIA DỮ LIỆU LÀM 2 PHẦN:
TRAINING VÀ TESTING
import pandas as pd
from tensorflow import keras
from sklearn.model_selection import train_test_split

# 1.1. Đọc file diabetes.csv sử dụng pandas
dataset = pd.read_csv('Dataset/diabetes.csv')

# 1.2. Chia dữ liệu: train (80%) and test (20%)
X = df.iloc[:, :-1].values   # tất cả các dòng, trừ cột cuối
y = df.iloc[:, -1].values    # tất cả các dòng, chỉ lấy cột cuối

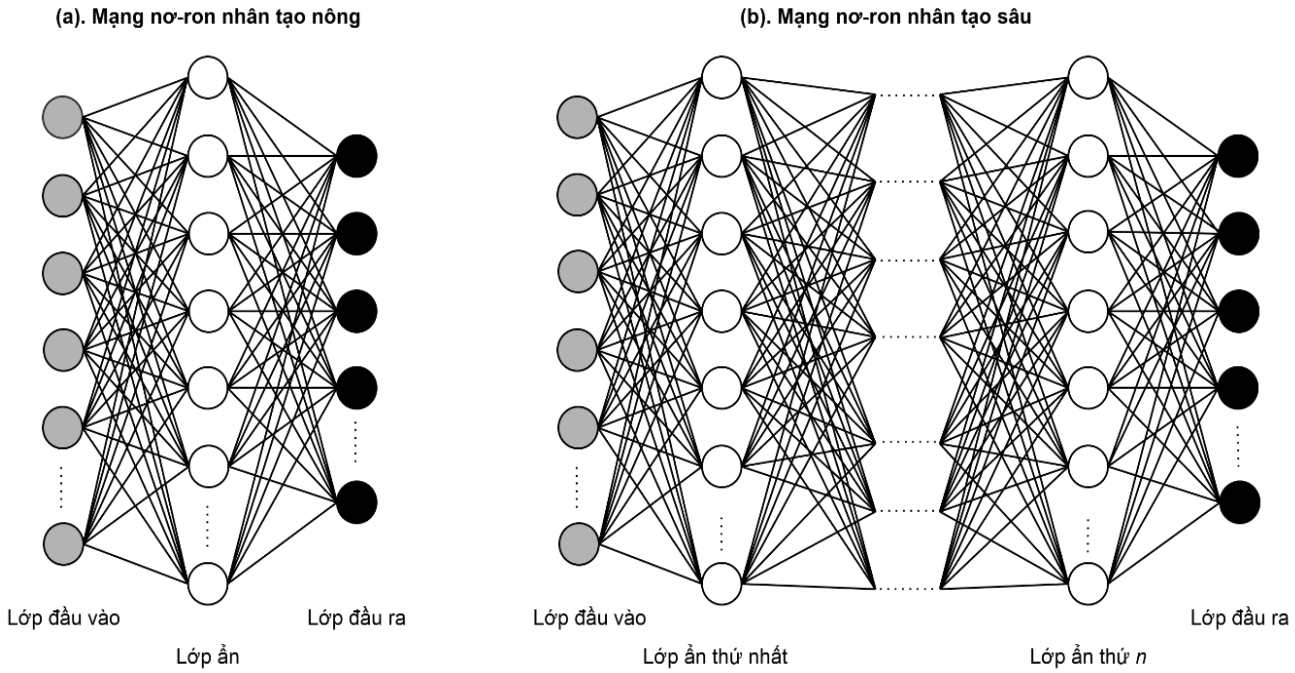
# Chia ngẫu nhiên
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 1.3. Mã hóa One-hot (One-hot encoding)
y_train_onehot = keras.utils.to_categorical(y_train, 2)
y_test_onehot = keras.utils.to_categorical(y_test, 2)

# Finished
print("Number of Training samples:", len(X_train))
print("Number of Testing samples:", len(X_test))
```

III. Phát triển mạng nơ-ron

- Bước 2: Xây dựng mạng nơ-ron: Tùy theo bài toán, dữ liệu ta xây dựng mạng nơ-ron nông hay sâu?



III. Phát triển mạng nơ-ron

- Bước 2: Xây dựng mạng nơ-ron:

```
# BƯỚC 2: XÂY DỰNG MẠNG NƠ RON
from tensorflow.keras.layers import Dense
from tensorflow.keras.models import Sequential

# 3.1. Xây dựng mạng nơ-ron với 1 lớp ẩn
def Diabetes_Network():
    # Khởi tạo mạng
    model = Sequential()

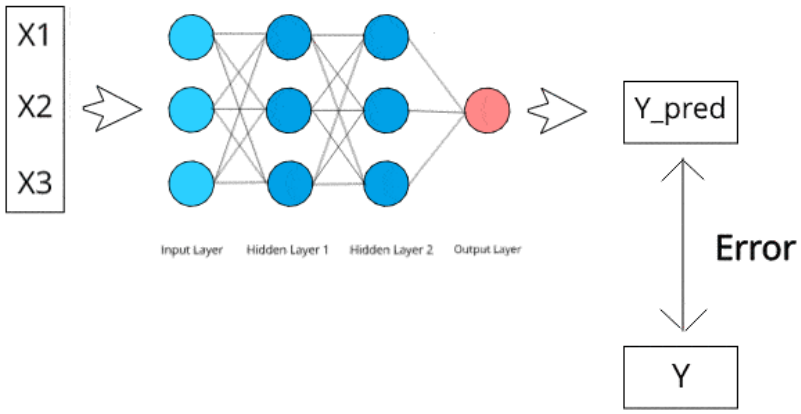
    # Xây dựng mạng: 1 lớp đầu vào + 1 lớp ẩn + 1 lớp đầu ra
    model.add(Dense(8, input_dim=8, activation='relu'))
    model.add(Dense(2, activation='softmax'))

    # Return
    return model

# 3.2. Hiển thị thông tin model (tuỳ chọn)
model = Diabetes_Network()
model.summary()
```

III. Phát triển mạng nơ-ron

- Bước 3: Lựa chọn hàm mất mát và tối ưu hoá:
  - Hàm mất mát (Loss function): Đo lường mức độ sai khác giữa dự đoán của mô hình và giá trị thực tế (ground truth).
  - Hàm mất mát phổ biến cho các bài toán phân loại: *Cross-entropy loss (CE), binary cross-entropy loss function (BCE), Categorical cross-entropy loss CCE).*
  - Hàm mất mát phổ biến cho các bài toán hồi quy: *MAE, MSE*.



III. Phát triển mạng nơ-ron

- Bước 3: Lựa chọn hàm mất mát và tối ưu hoá:
  - Tối ưu hóa (Optimization) là quá trình điều chỉnh trọng số và bias trong mạng để giảm giá trị hàm mất mát.
  - Thường dùng các thuật toán tối ưu như *Gradient Descent, SGD, Adam, RMSprop…*
  - Mục tiêu: Tìm bộ tham số tối ưu để mô hình học được quan hệ tốt nhất giữa dữ liệu đầu vào và đầu ra.

III. Phát triển mạng nơ-ron

- Bước 3: Lựa chọn hàm mất mát và tối ưu hoá:

```
# BƯỚC 3: LỰA CHỌN HÀM MẤT MÁT VÀ TỐI ƯU HOÁ
# Compile model trước khi training (có nhiều tuỳ chọn) (Xem code). Một số
tham số:
# loss: mean_squared_error/binary_crossentropy/categorical_crossentropy
# optimizer: SGD/RMSprop/Adam
    # SGD: Stochastic gradient descent, with support for momentum.
    # RMSprop: Adaptive learning rate optimization method proposed by Geoff
Hinton.
    # Adam: Adaptive Moment Estimation (Adam) that also uses adaptive
learning rates.
# metrics: accuracy

model.compile(loss='binary_crossentropy', optimizer='adam',
metrics=['accuracy']) # Hoặc
# model.compile(loss='categorical_crossentropy',
optimizer=keras.optimizers.Adam(learning_rate=0.0002),
metrics=['binary_accuracy'])
```

III. Phát triển mạng nơ-ron

- Bước 4: Huấn luyện mô hình:
  - Huấn luyện: Giúp mô hình học toàn bộ tập dữ liệu huấn luyện nhiều lần (epochs).
  - Huấn luyện mô hình nhiều epochs cho phép mô hình tối ưu tham số, giảm lỗi và tăng khả năng khái quát hóa.

```
# BƯỚC 4: HUẤN LUYỆN (TRAINING MODEL)
# 4.1. Training model (có nhiều tuỳ chọn). Một số tham số:
# epochs: Số lần training model.
# batch_size: Kích thước dữ liệu cho training.

# model.fit(X_train, y_train_onehot, epochs=10, batch_size=32) # Hoặc
history = model.fit(X_train, y_train_onehot, validation_split=0.2,
epochs=10, batch_size=16)

# 4.2. Lưu weight và model
# model.save_weights("modelNN_weight.hdf5")
model.save('modelNN_model.h5')
```

III. Phát triển mạng nơ-ron

- Bước 4: Huấn luyện mô hình: chú ý 4 tham số:
  - `X_train, y_train_onehot`: .
  - `validation_split:`
  - `epochs:`
  - `batch_size:`

III. Phát triển mạng nơ-ron

- Bước 5: Đánh giá và triển khai mô hình:
  - Đánh giá mô hình: Đánh giá hiệu quả của việc học dữ liệu.
  - Kết quả trên tập test phản ánh mức độ mô hình có thể dự đoán chính xác với dữ liệu mới, tức là độ tin cậy khi áp dụng trong thực tế.

```
# BƯỚC 5: ĐÁNH GIÁ (TESTING) MODEL
import numpy as np
from sklearn.metrics import accuracy_score, precision_score, recall_score,
f1_score

# 5.1. Hiển thị kết quả (accuracy) trên tập testing
test_loss, test_acc = model.evaluate(X_test, y_test_onehot)
print('Test loss:', test_loss)
print('Test accuracy:', test_acc)
```

III. Phát triển mạng nơ-ron

- Bước 5: Đánh giá và triển khai mô hình:
  - Triển khai: Xây dựng các ứng dụng thực tế sử dụng mô hình đã được huấn luyện…

```
# BƯỚC 5: SỬ DỤNG TRAINED MODEL
from tensorflow.keras.models import Sequential, Model, load_model,
Sequential

# 5.1. Load lại model
model_loaded = load_model('modelNN_model.h5')

# 5.2. Hiển thị các kết quả khác (Accuracy, Precision, Recall, F1 score)
trên tập testing => Như bước 4.
y_test_predicted = model.predict(X_test)
y_test_predicted=np.argmax(y_test_predicted, axis=1)
```

IV. Xây dựng mạng nơ-ron

- Xây dựng mạng nơ-ron cho bài toán dự đoán mắc bệnh tiểu đường:

```
# BƯỚC 1: ĐỌC DỮ LIỆU TỪ FILE diabetes.csv VÀ CHIA DỮ LIỆU LÀM 2 PHẦN: TRAINING VÀ TESTING
import pandas as pd
from tensorflow import keras
from sklearn.model_selection import train_test_split

# 1.1. Đọc file diabetes.csv sử dụng pandas
dataset = pd.read_csv('Dataset/diabetes.csv')

# 1.2. Chia dữ liệu: train (80%) and test (20%)
X = df.iloc[:, :-1].values   # tất cả các dòng, trừ cột cuối
y = df.iloc[:, -1].values    # tất cả các dòng, chỉ lấy cột cuối

# # Chia ngẫu nhiên
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

# 1.3. Mã hóa One-hot (One-hot encoding)
y_train_onehot = keras.utils.to_categorical(y_train, 2)
y_test_onehot = keras.utils.to_categorical(y_test, 2)

# Hiển thị số bản ghi tập train/test
print("Number of Training samples:", len(X_train))
print("Number of Testing samples:", len(X_test))
```

IV. Xây dựng mạng nơ-ron

- Xây dựng mạng nơ-ron cho bài toán dự đoán mắc bệnh tiểu đường:

```
# BƯỚC 2: XÂY DỰNG MẠNG NƠ RON
from tensorflow.keras.layers import Dense
from tensorflow.keras.models import Sequential

# 2.1. Xây dựng mạng nơ-ron với 1 lớp ẩn
def Diabetes_Network():
    # Khởi tạo mạng
    model = Sequential()

    # Xây dựng mạng: 1 lớp đầu vào + 1 lớp ẩn + 1 lớp đầu ra
    model.add(Dense(8, input_dim=8, activation='relu'))
    model.add(Dense(2, activation='softmax'))

    # Return
    return model

# 2.2. Hiển thị thông tin model (tuỳ chọn)
model = Diabetes_Network()
model.summary()
```

IV. Xây dựng mạng nơ-ron

- Xây dựng mạng nơ-ron cho bài toán dự đoán mắc bệnh tiểu đường:

```
# BƯỚC 3: LỰA CHỌN HÀM MẤT MÁT VÀ TỐI ƯU HOÁ
# Compile model trước khi training (có nhiều tuỳ chọn) (Xem code). Một số tham số:
# loss: mean_squared_error/binary_crossentropy/categorical_crossentropy
# optimizer: SGD/RMSprop/Adam
    # SGD: Stochastic gradient descent, with support for momentum.
    # RMSprop: Adaptive learning rate optimization method proposed by Geoff Hinton.
    # Adam: Adaptive Moment Estimation (Adam) that also uses adaptive learning rates.
# metrics: accuracy/binary_accuracy

model.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy']) # Hoặc
# model.compile(loss='categorical_crossentropy', optimizer=keras.optimizers.Adam(learning_rate=0.0002), metrics=['binary_accuracy'])
```

IV. Xây dựng mạng nơ-ron

- Xây dựng mạng nơ-ron cho bài toán dự đoán mắc bệnh tiểu đường:

```
# BƯỚC 4: HUẤN LUYỆN (TRAINING MODEL)
from tensorflow import keras
import matplotlib.pyplot as plt

# 4.1. Training model (có nhiều tuỳ chọn). Một số tham số:
# epochs: Số lần training model.
# batch_size: Kích thước dữ liệu cho training.

# model.fit(X_train, y_train_onehot, epochs=10, batch_size=32) # Hoặc
# model.fit(X_train, y_train_onehot, validation_split=0.2, epochs=10, batch_size=32) # Hoặc
# history = model.fit(X_train, y_train_onehot, validation_split=0.2, epochs=10, batch_size=32, callbacks=[checkpointer]) # Hoặc
# history = model.fit(X_train, y_train_onehot, validation_split=0.2, epochs=10, Dropout=0.2, batch_size=16, callbacks=[checkpointer]) # Hoặc
history = model.fit(X_train, y_train_onehot, validation_split=0.2, epochs=10, batch_size=16)

# 4.2. Lưu weight và model
# model.save_weights("modelNN_weight.hdf5")
model.save('modelNN_model.h5')

# 4.3. Hiển thị Training and Validation Loss
plt.figure(figsize=(8, 6))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Training and Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.show()
```

IV. Xây dựng mạng nơ-ron

- Xây dựng mạng nơ-ron cho bài toán dự đoán mắc bệnh tiểu đường:

```
# BƯỚC 5: ĐÁNH GIÁ (TESTING) MÔ HÌNH
import numpy as np
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# 5.1. Hiển thị kết quả (accuracy) trên tập testing
test_loss, test_acc = model.evaluate(X_test, y_test_onehot)
print('Test loss:', test_loss)
print('Test accuracy:', test_acc)
```

IV. Xây dựng mạng nơ-ron

- Xây dựng mạng nơ-ron cho bài toán dự đoán mắc bệnh tiểu đường:

```
# BƯỚC 5: TRIỂN KHAI (SỬ DỤNG) MÔ HÌNH ĐÃ ĐƯỢC HUẤN LUYỆN
from tensorflow.keras.models import Sequential, Model, load_model, Sequential

# 5.1. Load lại model
model_loaded = load_model('modelNN_model.h5')

# 5.2. Hiển thị các kết quả khác (Accuracy, Precision, Recall, F1 score) trên tập testing => Như bước 4.
y_test_predicted = model.predict(X_test)
y_test_predicted=np.argmax(y_test_predicted, axis=1)

print('Accuracy:', accuracy_score(y_test,y_test_predicted))
print('Precision:', precision_score(y_test,y_test_predicted))
print('Recall:', recall_score(y_test, y_test_predicted))
print('F1 score:', f1_score(y_test, y_test_predicted))
```

IV. Xây dựng mạng nơ-ron

- Xây dựng mạng nơ-ron cho bài toán dự đoán mắc bệnh tiểu đường:

```
# BƯỚC 6: SỬ DỤNG TRAINED MODEL DỰ ĐOÁN MỘT BỆNH NHÂN CỤ THỂ
# 6.1. Load lại model
model_loaded = load_model('modelNN_model.h5')

# 6.2. Dự đoán 1 bệnh nhân cụ thể
features = [13.1976, 0, 50.0, 110.1, 10, 35.7, 10, 10]
results = model_loaded.predict([features])
# results = np.argmax(results, axis=1)
results = np.argmax(results, axis=1)
print("Dự đoán: ", results)

# 6.3. Xây dựng ứng dụng web (cần IDE như: Pycharm...)
```

Tài liệu tham khảo

- [1]. https://medium.com/
- [2]. https://towardsdatascience.com/
- [3]. https://machinelearningcoban.com/
- [4]. https://github.com/mrdbourke/pytorch-deep-learning
- [5]. Huang, Shih-Chia, and Trung-Hieu Le. *Principles and labs for deep learning*. Academic Press, 2021.





Thank You !

Question?